

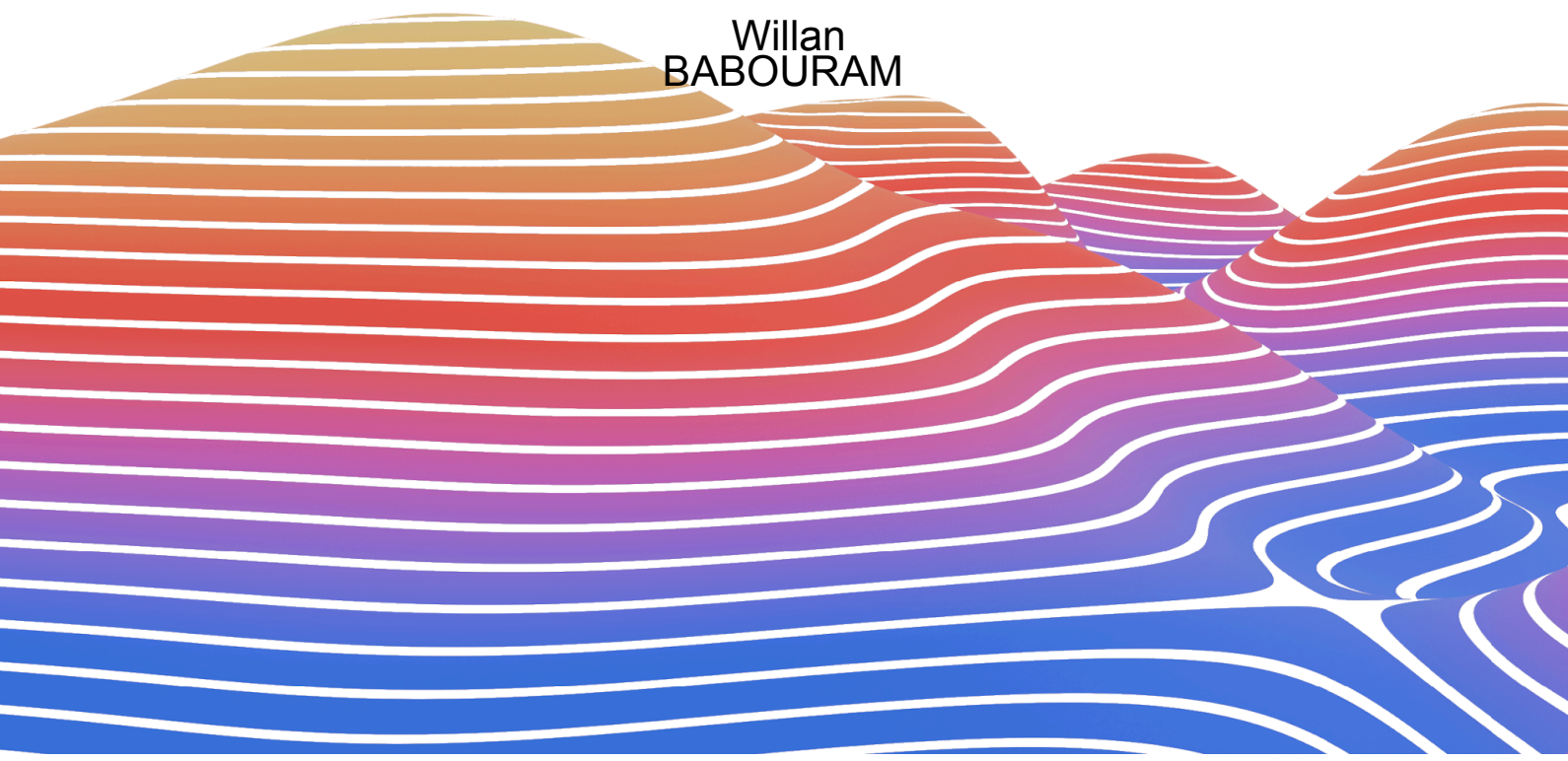
BUT1 INFO - Groupe D1

SAE 1.01

Report

Anas
KIOUAZ

Willan
BABOURAM



Contents

1 What did we do?	1
1.1 Manually written glossaries	1
1.2 Glossary parsing	1
1.3 Classification of the news into categories	2
1.4 Automatically created glossaries	4
1.5 k-NN implementation	5
1.5.1 By content proximity	5
1.6 Processing time improvement	5
2 Results	5
2.1 Accuracy with score	5
3 Complexiter	6
3.1 CalculScores	6
3.2 Score	6

As a reminder, the goal of this **SAÉ** was to develop a program that categorizes news articles into the following topics:

• **Science** • **Culture** • **Economy** • **Politics** • **Sports**

To achieve this, we implemented a **supervised machine learning algorithm** that assigns a weight (from 1 to 3) to each word appearing in the news of a specific category. These word lists, referred to as “**glossaries**” in this report, were built in two phases:

1. **Manual Compilation** – At first, we manually created the glossaries by selecting relevant words based on subjective judgment.

2. **Automatic Creation** – Later, we generated the glossaries automatically by calculating a score for each word, based on its frequency in its category compared to others in a training dataset (**depeches.txt**). These scores were then converted into weights and saved in separate files.

The program then uses these glossaries to analyze unclassified news articles and determine the most appropriate category—the one with the highest score.

Throughout the project, we not only focused on the required implementation but also tested different approaches to improve the program’s **accuracy** and **processing speed**. All these stages are explained in detail later in this document.

1 What did we do?

1.1 Manually written glossaries

We then assigned a weight to each of these words.

This first step allowed us to test the initial part of the SAÉ without depending on the automatic glossary generation, which was added later.

For example, the top five words in the SPORTS category are:

```
1 Médailles:3
2 Jeux:3
3 paralympiques:2
4 athlètes:3
```

Fig. 1: The 5 first lines of the sports.txt file

1.2 Glossary parsing

Our first step was to implement the **initLexique** procedure in the **Category** class to manage reading the glossary files. These files follow a specific format, as outlined in the instructions.

To handle them correctly, we adapted the file-reading method from the **lectureDepeches** procedure in the **Classification** class.

Once the code was completed, we tested it by printing the words from the **SPORTS**

category glossary.

```
using getlexique().get(i).afficher()
```

```
1 Médailles : 3
2 Jeux : 3
3 paralympiques : 2
4 athlètes : 3
```

Fig. 2: Output of the program, printing the glossary of the **SPORTS** category

1.3 News' score calculation for a category

The score of a news article in a specific category is calculated by summing the weights of its words based on that category's glossary.

To achieve this, we first needed a function to retrieve a word's weight from a category's glossary. For this, we created the **entierPourChaine** function in the **UtilitairePaireEntierChaine** class.

To test this function, we modified our program to ask the user for a word and display its weight in the **SPORTS** category.

```
1 Entrez un mot : athlètes
2 poids de athlètes est 3 dans la catégorie sport
```

Fig. 3: Output of the program when the user writes the word "athlètes"

Next, we needed a function to calculate the sum of the weights of the words in a specific news article. To achieve this, we implemented the score function in the **Category** class.

To test this function, we modified our program to print a specific news article, list every word from its content with a weight greater than 0 in the **SPORTS** category, and finally display the score calculated using the score function.

```
1 -----
2 dépêche 401
3 date:08/09/2024
4 catégorie:SPORT
5 En direct, Jeux paralympiques :Nélia Barbosa médaillée d'argent
  t en canoë ; Rosario Murcia-Gangloff à la quatrième place du
  marathon. Pour cette dernière journée des Jeux paralympiques, la
  France pouvait espérer trois ultimes médailles. Aurélie Aubert e
  t Tanguy de La Forest seront les porte-drapeaux de la délégation
  française ce soir pour la cérémonie de clôture qui démarrera à 2
  0 h 30.
6 -----
7 score de la dépêche pour la catégorie sport est 4
```

Fig. 4: Output of the program, showing the 400th news, its word with a weight > 0 and its score for the **SPORTS** category

1.3 Classification of the news into categories

To classify a news between our categories, we need to:

1. calculate its score in each category,
2. choose the category where it gets the highest score.

We classify a news into the category where it scores the most. We implemented the `chaineMax` function in the `UtilitairePaireChaineEntier` class to do this.

```
1 La dépêche numéro 401 a
2 un score de 4 pour la catégorie sport
3 un score de 0 pour la catégorie politique
4 un score de 0 pour la catégorie economie
5 un score de 0 pour la catégorie culture
6 un score de 0 pour la catégorie sciences
```

Fig. 5: Output of the program, showing the score of the 463rd news in each category, and the category where it gets the highest score (SPORTS) using `chaineMax`

To calculate the percentage of correct answers for each category, we determined the number of correct classifications per category and divided it by the total number of news articles in that category. For the global percentage of correct answers, we summed up the correct classifications across all categories and divided the result by the total number of news articles.

We implemented the `classementDepeches` procedure in the `Classification` class to:

- Classify each news article into the category with the highest score,
- Calculate the percentage of correct answers for each category,
- Calculate the global percentage of correct answers,
- Save the results in the specified format to a text file.

After testing, our procedure successfully generated a file in the required format. Below is an example of a result produced by the procedure:

```
1 001:economie
2 [...]
3 500:sport
4 sport:90%
5 politique:29%
6 economie:23%
7 culture:37%
8 sciences:21%
8 MOYENNE:40.0%
```

Fig. 6: Content of the file generated by `classementDepeches`

1.4 Automatically created glossaries

To automatically generate a glossary for a group based on its articles, we simply need, for each term in the group's article, to compute its value based on its occurrence (e.g. increase the value by 1 if we encounter it in the group, and decrease it by 1 if it appears in another group), and determine a weight based on this value.

We implemented `initDico` in the `Classification` class to build a list of distinct terms in the articles of the group, then we created `calculScores` to calculate the value of a term based on how often it appears in the group's articles.

At this point, we had to figure out a method to assign a weight based on a value, and we decided to assign:

- 3 for terms with a value over 20,
- 2 for terms with a value between 10 and 5,
- 1 for terms with a value greater than 0.

Finally, we developed the `generationLexique` function to build the list of terms, assign values and weights for each group, and then save all of this into a file. Here's an example of a glossary generated by this function:

```
1 001:economie
2 [...]
3 500:sport
4 sport:87%
5 politique:82%
6 economie:54%
7 culture:66%
8 sciences:56%
9 MOYENNE:69.0%
```

Fig. 8: Content of the answer file using auto-generated glossaries

```
1 rine:2
2 pen:2
3 dent:1
4 s:1
5 être:1
6 entene:1
7 avec:1
8 emnuel:2
```

Fig. 7: Content of the culture.txt file generated by generationLexique

1.5 k-NN implementation

In order to increase the success rate, two implementation were made around the k-NN method, which unlike our previous implementations don't choose the result based on the best score of categories but on the "proximity" with previously classified news.

1.5.1 By content proximity

A first simple implementation was made based on what was asked in the document, which said that in order to compare two news we will base our proximity on their content by counting the similar word, such as the more there are commonalities the nearest are the news.

In the end, the results were even worse than our very first basic implementation.

1.6 Processing time improvement

After attempting to improve result accuracy, we focused on reducing the processing time of glossary generation.

To achieve this, we sort the glossary words during initialization (in initDico): for each new word, we determine the index at which to add it to the glossary being created using binary search, and we replace all the sequential searches we previously implemented with a binary search implementation

2 Results

2.1 Accuracy with score

Sucess Pourcent

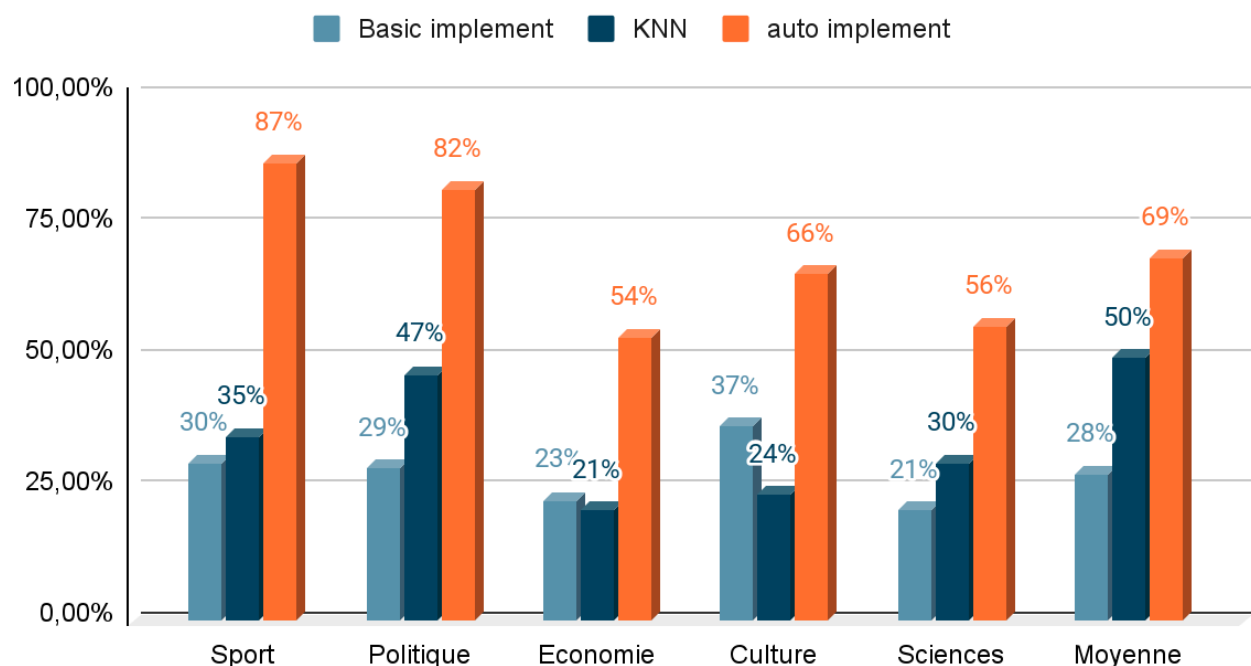
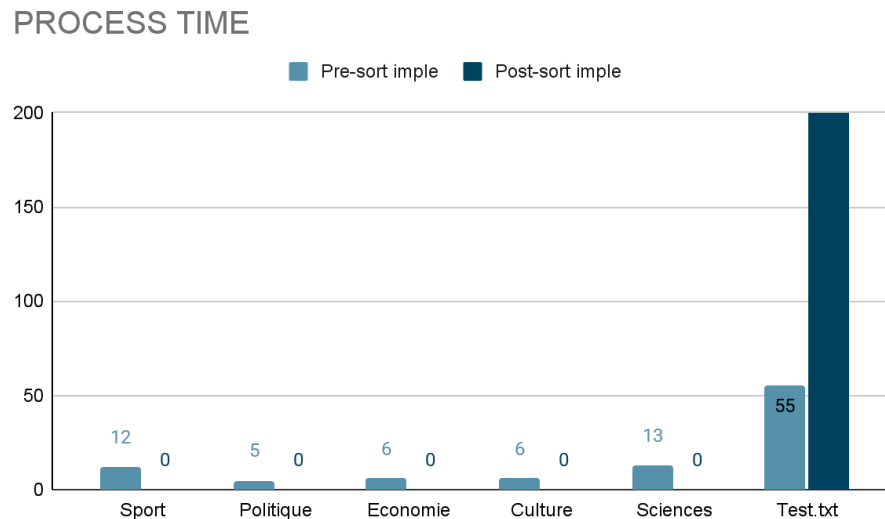


Fig. 11: Accuracy rates during classification with test.txt

In the **Sports** category, the results are more precise compared to other categories. However, there are also more false positives classified in this category, leading us to deduce that it is simply the most likely to appear regardless of the correct category. This is also due to the fact that Sports contain specific terms predominantly used in that category.

On the other hand, in the **Env/Sci** category, the vocabulary is more general, with words appearing in multiple categories, making accurate classification more challenging.

Processing time



3 Complexiter

3.1 CalculScores

In the **calculScores** procedure, we calculate the frequency of each word in all articles within its category and other categories to compute the score for that word. Given that w represents the total number of words in all articles, and n represents the number of words in the glossary for the category:

- When using **binary search**, the worst-case time complexity of the scoring procedure is approximately $-w \log(n)$.
- When using **sequential search**, the worst-case time complexity of the scoring procedure is approximately $-wn$.

The overall complexity of this procedure aligns closely with the score computation process. Essentially, we multiply the complexity of the search algorithm being used by the total number of words across all articles, rather than just the number of words in a single article.

Here the calculeScores count post sort is 17567
(score)count post sort 49010